REACT NATIVE • EXPO ROUTER

UYGULAMA FÖYÜ

Arıza Takip + Mini Ürün Kataloğu + Öğrenci Ders Programı

Ders: Mobil Programlama | Tarih: 27 Ekim 2026 | Düzey: Ön lisans

Ad Soyad: ..... Numara: .....

Amaç, araçlar ve teslim

Amaç: Bir mobil uygulamayı ekranlara ve ortak bileşenlere ayırmak; Expo Router ile gezinme ve dinamik rota üzerinden detay ekranı oluşturmak.

Araçlar: Node.js, Expo, VS Code, Expo Go/Android emülatör veya web. Dil: JavaScript (JSX). Süre: 160 dakika.

Teslim: Proje klasörü veya GitHub bağlantısı, uygulama ekran görüntüleri, test tablosu ve 5–8 cümlelik kısa değerlendirme. Gerekli örnek veriler haricinde internet/veritabanı zorunlu değildir.

Çalışma akışı

Görev	Süre	Çıktı
1. Arıza Takip – yeniden düzenleme	45 dk	Liste ve detay arası gezinme
2. Mini Ürün Kataloğu	45 dk	Ürün kartları + detay
3. Ders Programı – farklı senaryo	35 dk	Ders listesi + detay
4. Test, hata düzeltme, teslim	35 dk	Test kayıtları ve rapor

UYGULAMA 1 | Arıza Takip uygulamasına navigasyon

Senaryo: Önceki hafta hazırladığınız Teknik Destek / Arıza Takip projesini devam ettirin. Önceki ekranları silmeyin; kodlarınızı farklı ekranlara ve bileşenlere ayırın. Öncelikle uygulamanın çalışan bir kopyasını alın.

A. Gereksinimler

- Ana ekrandan “Arıza Listesi” ekranına geçiş olsun.
- Arıza listesinde en az 3 kayıt bulunsun; her kayıt TicketCard bileşeni ile gösterilsin.
- Her kartın detay bağlantısı, ilgili arıza ID’si ile /ariza/[id] ekranını açsın.
- Detay ekranında başlık, konum, durum ve geri butonu yer alsın.
- StatusBadge bileşeni durum değerine göre görsel farklılık oluştursun.
- Yeni Arıza ekranı navigasyonda açılsın. Önceki hafta çalışan form varsa koruyun; yoksa bu ders için temel form yeterlidir.

B. İzlenecek adımlar

1. Var olan uygulamanın dosya yapısını inceleyin. Router’ın kurulu olup olmadığını denetleyin.
2. app/_layout.jsx içinde Stack yapısını oluşturun.
3. Tekrarlanan arıza kartını components/TicketCard.jsx dosyasına taşıyın.
4. Örnek verileri data/tickets.js dosyasında tutun (veya mevcut ortak veri kaynağınızı koruyun).
5. Liste ekranında FlatList kullanın ve onPress ile dinamik rotaya geçin.
6. Dinamik detay ekranında useLocalSearchParams ile id okuyun; doğru arızayı bulun.

7. StatusBadge ekleyin ve en az üç farklı durumu renklendirin.
8. Uygulamayı çalıştırın, tüm ekran geçişlerini deneyin.

C. Öğrenci müdahalesi: Kodları bire bir kopyalamadan tamamlayın

- TicketCard başlık, konum ve durum gösterecek. Bu alanların yerleşimini siz belirleyin.
- Bekliyor = turuncu, İşlemde = mavi, Çözüldü = yeşil olacak biçimde StatusBadge yazın.
- Olmayan /ariza/999 adresine gidildiğinde çökme yerine anlamlı uyarı gösterin.
- Detaydan geri gelindiğinde liste ekranı tekrar kullanılabilsin.

UYGULAMA 2 | Mini Ürün Kataloğu

Senaryo: Küçük bir teknoloji mağazası ürünlerini mobil katalog üzerinden göstermek istiyor. Bu uygulama ayrı Expo projesi veya ayrılmış çalışma klasörü olarak hazırlanmalıdır.

A. Beklenen ekranlar

Ekran	Beklenen işlev
Ürün listesi (/)	En az 5 ürün; kartta ad, fiyat, kategori ve stok
Ürün detayı (/urun/[id])	Seçilen ürüne ait ad, fiyat, açıklama, stok bilgisi
Geri gezinme	Detaydan ürün listesine geri dönme

B. Örnek veri sözleşmesi

data/products.js – başlangıç verisi

```
export const products = [
  { id: 'p1', name: 'Kablosuz Fare', price: 450, category: 'Aksesuar', stock: 8,
    description: 'Bluetooth bağlantılı ergonomik fare' },
  { id: 'p2', name: 'Mekanik Klavye', price: 1800, category: 'Aksesuar', stock: 0,
    description: 'Mekanik tuşlu klavye' },
];
// Siz en az 3 ürün daha ekleyin.
```

C. Öğrencinin tamamlayacağı görevler

- ProductCard bileşeni tanımlayın; props üzerinden tek ürün alsın.
- Ürün listesinde FlatList kullanın.
- Ürüne basınca router.push ile /urun/<id> ekranını açın.
- [id].jsx ekranında useLocalSearchParams kullanın.
- Ürün bulunmadığında hata ekranı gösterin.
- Stok 0 ise “Stokta yok”, diğer durumda “Stokta” yazdırın.
- Kendi tasarım kararınızı ekleyin (renk/typography/yerleşim) ve neden seçtiğinizi açıklayın.

UYGULAMA 3 | Öğrenci Ders Programı

Senaryo: Öğrenci haftalık derslerini listelemek ve seçtiği dersin detaylarını görüntülemek istiyor. Bu uygulamada tam kod verilmez; klasör yapısı ve veri modeli öğrencinin kararıdır.

Gereksinimler

- En az 4 ders tanımlayın: id, ders adı, gün, saat, sınıf, öğretim elemanı.
- Ana sayfada gün veya ders filtresi bulunması tercih edilir (zorunlu değil).
- CourseCard bileşeni kullanın.
- Kart seçildiğinde dinamik ders detay ekranına yönlendirin.

- Detay ekranında tüm ders bilgileri ve geri seçeneği yer alsın.
- En az bir hatalı ID denemesinde kontrollü bilgi mesajı gösterin.

Örnek veri (kod yerine örnek kayıt)

ID	Ders	Gün	Saat	Sınıf
d1	Mobil Programlama	Salı	09.00	Lab 2
d2	JavaScript	Çarşamba	13.00	Lab 1
d3	Web Tasarımı	Perşembe	10.00	Derslik 3
d4	Veritabanı	Cuma	11.00	Lab 4

Ek çalışma | Sekmeli Görev Takip (isteğe bağlı)

Uygulamayı bitirenler için ayrı bir projede Tabs navigasyonu kurun: Görevler, Tamamlananlar ve Profil sekmeleri. Her sekme ayrı dosya/rota olmalıdır. Görev kartları ortak bileşenden üretilmelidir.

Test senaryoları ve hata ayıklama

No	İşlem	Beklenen sonuç	Sonuç
T1	Ana sayfadan Arıza Listesine git	Liste görünür	
T2	101 numaralı arızayı aç	101 detayları görünür	
T3	Detaydan geri dön	Liste yeniden görünür	
T4	Bilinmeyen arıza ID aç	Uyarı görünür; uygulama çökmez	
T5	Stok=0 olan ürünü aç	“Stokta yok” yazısı görünür	
T6	İkinci bir ürüne geç	Doğru ürün ayrıntıları görünür	
T7	Farklı bir derse bas	Seçilen dersin bilgileri görünür	

Yapay zekâ kullanım sınırları

- Yapay zekâdan sözdizimi, hata mesajı veya tek bir bileşen hakkında yardım alınabilir.
- Ekran akış şemasını ve bileşen ayrıştırma kararını öğrenci kendisi vermelidir.
- En az bir hata öğrencinin kendi açıklamasıyla kaydedilmelidir: sorun – neden – çözüm.
- Teslimde “Yapay zekâdan nerede yararlandım?” başlığıyla 2–3 cümle yazılmalıdır.

Değerlendirme rubriği

Ölçüt	Puan
Proje yapısı ve dosya düzeni	15
En az 2 yeniden kullanılabilir bileşen	20
Navigasyon ve ekran geçişleri	25
Dinamik rota ve doğru veri gösterimi	20
Arayüz / kullanılabilirlik	10
Test ve hata düzeltme	10

TOPLAM	100

Öğrenci kısa raporu

1. Hangi bileşenleri yeniden kullanılabilir hâle getirdim? .....
2. Hangi navigasyon işlemlerini kullandım? .....
3. Karşılaştığım bir hata ve çözümü: .....
4. Yapay zekâdan yararlandığım alanlar: .....
5. Uygulamanın geliştirilmesi için bir önerim: .....

Teslim kontrol listesi

- Proje açılıyor ve Expo üzerinde çalışıyor.
- Tüm ekranların görüntüsü eklendi.
- Bileşenler ayrı dosyalarda bulunuyor.
- Test tablosu işaretlendi ve hata notu yazıldı.
- Rapor tamamlandı.